

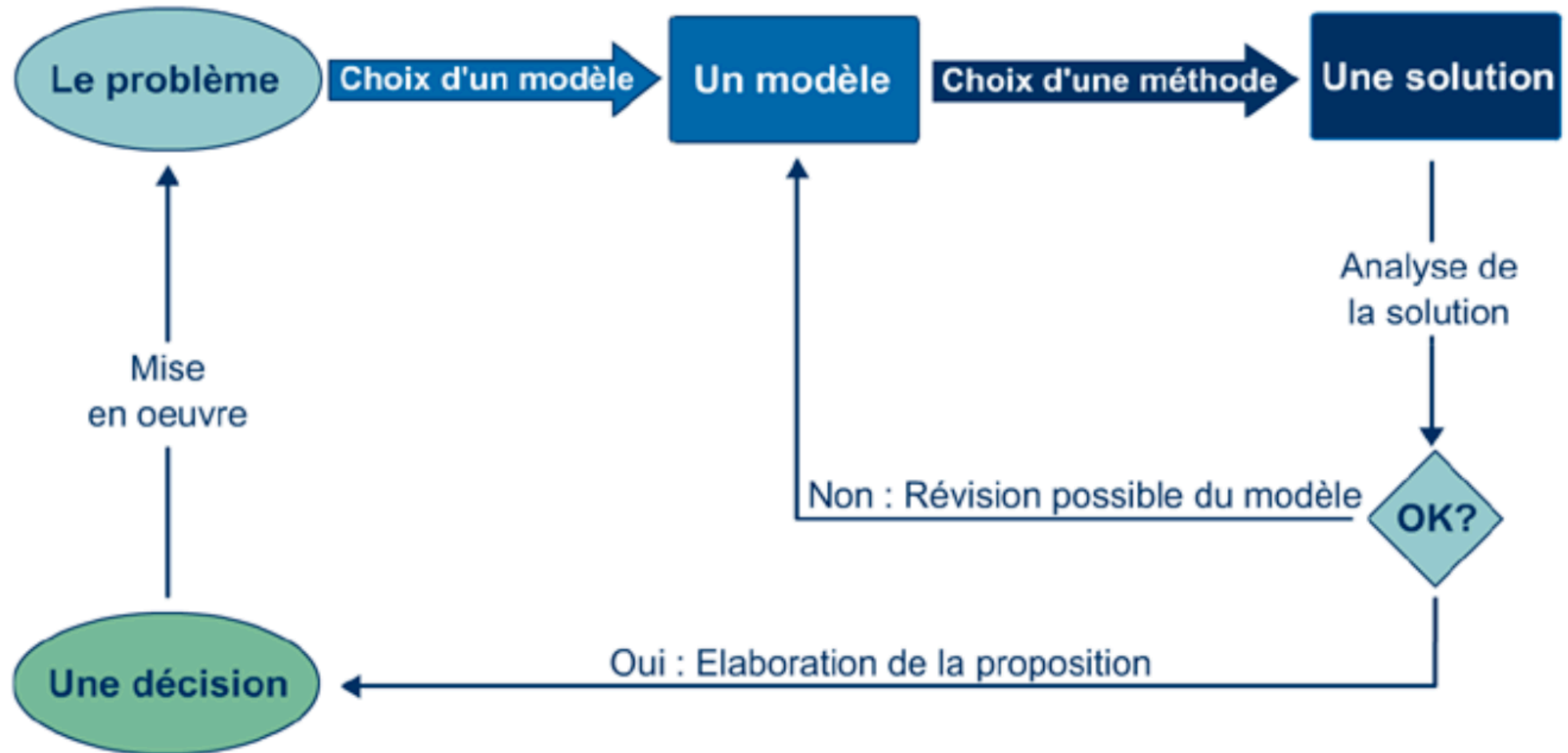
MeREA : Méthodes de Résolution Exactes et Approchées pour l'Optimisation Combinatoire

Maria Zrikem

GI2 ENSA de Marrakech

2026 - 2027

La démarche de l'aide à la décision



Ce module s'intéresse à des méthodes des classiques employées en optimisation combinatoire.

Partie 1 : Les méthodes **exactes**

Partie 2 : Les méthodes **approchées**

Partie 1 : Les méthodes **exactes**

Quelques problèmes d'Optimisation Combinatoire

Rappel sur la programmation linéaire (Postoptimalité)

Programmation linéaire en nombres entiers

Méthode de séparation et d'évaluation (Branch and Bound)

Méthode exacte pour le problème de transport

Méthode exacte pour le problème d'affectation

Méthode de programmation dynamique

Algorithmes de retour arrière (Backtracking)

Méthode (Branch and Cut) et Méthode de la génération de colonnes

Partie 2 : Les méthodes **approchées** (Heuristiques et Métaheuristiques)

- Introduction

 - Optimisation combinatoire

 - Les heuristiques constructives

- Les algorithmes de recherche locale

 - Éléments de base : Représentation et voisinage

 - Le GRASP : une première métaheuristique

- Etude de cas

 - Recherche locale appliquée au problème de l'arbre de Steiner

 - Recherche locale appliquée à la coloration de graphes

- La recherche Tabou et la Recherche à Voisinage Variable

- Etude de cas : une RVV vs RT pour l'optimisation du routage de câbles dans les installations de production d'électricité

- Le recuit simulé (simulated annealing)

- Algorithmes génétiques

- Les algorithmes de colonies de fourmis

Optimisation Combinatoire

Problème d'optimisation combinatoire

- Un problème d'optimisation combinatoire (OC) consiste à déterminer un plus grand (petit) élément dans un ensemble fini valué.
- La forme très générale d'un problème d'optimisation est :

$$\begin{cases} \min f(x) \\ x \in S \end{cases} \quad \text{Ou} \quad \begin{cases} \max f(x) \\ x \in S \end{cases}$$

- La famille S représente donc les solutions possibles (réalisable) du problème. Elle peut correspondre à un ensemble de très grande taille que l'on ne connaît que par des descriptions ou des propriétés théoriques qui ne permettent pas facilement son énumération

Problème d'optimisation combinatoire

L'homme envisage difficilement la multiplicité des combinaisons qui se présentent, même dans les moindres faits de la vie, lorsque plusieurs variables peuvent prendre, chacune, des états différents.

Exemple : Combien faut-il de temps à une famille de 8 personnes, prenant en commun deux repas journaliers, pour épuiser toutes les possibilités de se grouper autour de la table familiale.

OC : Problèmes faciles et problèmes difficiles

La classe P (Pbs faciles) : Un problème de décision appartient à la classe P si et seulement si il est possible de le résoudre à l'aide d'un algorithme déterministe en temps polynomial ($O(n^d)$).

La classe NP (Nondeterministic Polynomial) : est celle des problèmes de décision (i.e à réponse dans oui, non) pour lesquels il existe un algorithme déterministe polynomial qui permet de vérifier la validité d'une solution proposée du problème supposée à réponse oui. Un tel algorithme déterministe polynomial est appelé un vérifieur. La communauté scientifique admet généralement que $P \neq NP$ (sauf preuve du contraire)

OC : Problèmes faciles et problèmes difficiles

La classe NP-complets : Un problème NP-complets possède la priorité que tout problème dans NP peut être transformé (réduit) en celui-ci en temps polynomial. C'est à dire qu'un problème est NP-complets quand tous les problèmes appartenant à NP lui sont réductibles. Si on trouve un algorithme polynomial pour un problème NP-complets, on trouve alors automatiquement une résolution polynomiale de tous les problèmes de la classe NP.

La classe NP-difficiles : Un problème est NP-difficile s'il est plus difficile qu'un problème NP-complet, c'est à dire s'il existe un problème NP-complet se réduisant à ce problème par la réduction de Turing.

On ne connaît pas d'algorithmes polynomiaux pour ces problèmes et on n'arrive pas à démontrer la non-existence de tels algorithmes et s'il existe un algorithme polynomial pour un de ces problèmes, alors il existe des algorithmes polynomiaux pour toute la classe.

OC : Problèmes faciles et problèmes difficiles

L' « optimisation combinatoire » consiste à trouver la meilleure solution entre un nombre fini de choix. Autrement dit, à minimiser une fonction, avec ou sans contraintes, sur un ensemble fini de possibilités. Quand le nombre de combinaisons possibles devient exponentiel par rapport à la taille du problème, le temps de calcul devient rapidement critique : phénomène l'*explosion combinatoire*.

OC : Problèmes faciles et problèmes difficiles

	n = 10	n = 1000	n = 100000	
$O(n)$	n 10n 100n	10 100 1000	1000 10^4 10^5	10^5 10^6 10^7 } secondes
$O(n^2)$	n² 10n² 100n²	100 1000 10^4	10^6 10^7 10^8	10^{10} 10^{11} 10^{12} } heures
$O(n^3)$	n³ 10n³ 100n³	1000 10^4 10^5	10^9 10^{10} 10^{11}	10^{15} 10^{16} 10^{17} } années
$O(2^n)$	2ⁿ 10 · 2ⁿ 100 · 2ⁿ	1024 $> 10^4$ $> 10^5$	$> 10^{301}$ $> 10^{302}$ $> 10^{303}$	∞ ∞ ∞
$O(\lg n)$	lg n 10 lg n 100 lg n	3.3 33.2 332.2	9.9 99.6 996.5	16.6 166.1 1661.0

La difficulté de résolution d'un problème ayant un nombre exponentiel de solutions n'est donc pas un simple problème technique que des ordinateurs très puissants pourraient balayer. Il est nécessaire de contourner l'explosion combinatoire par des outils mathématiques et algorithmiques.

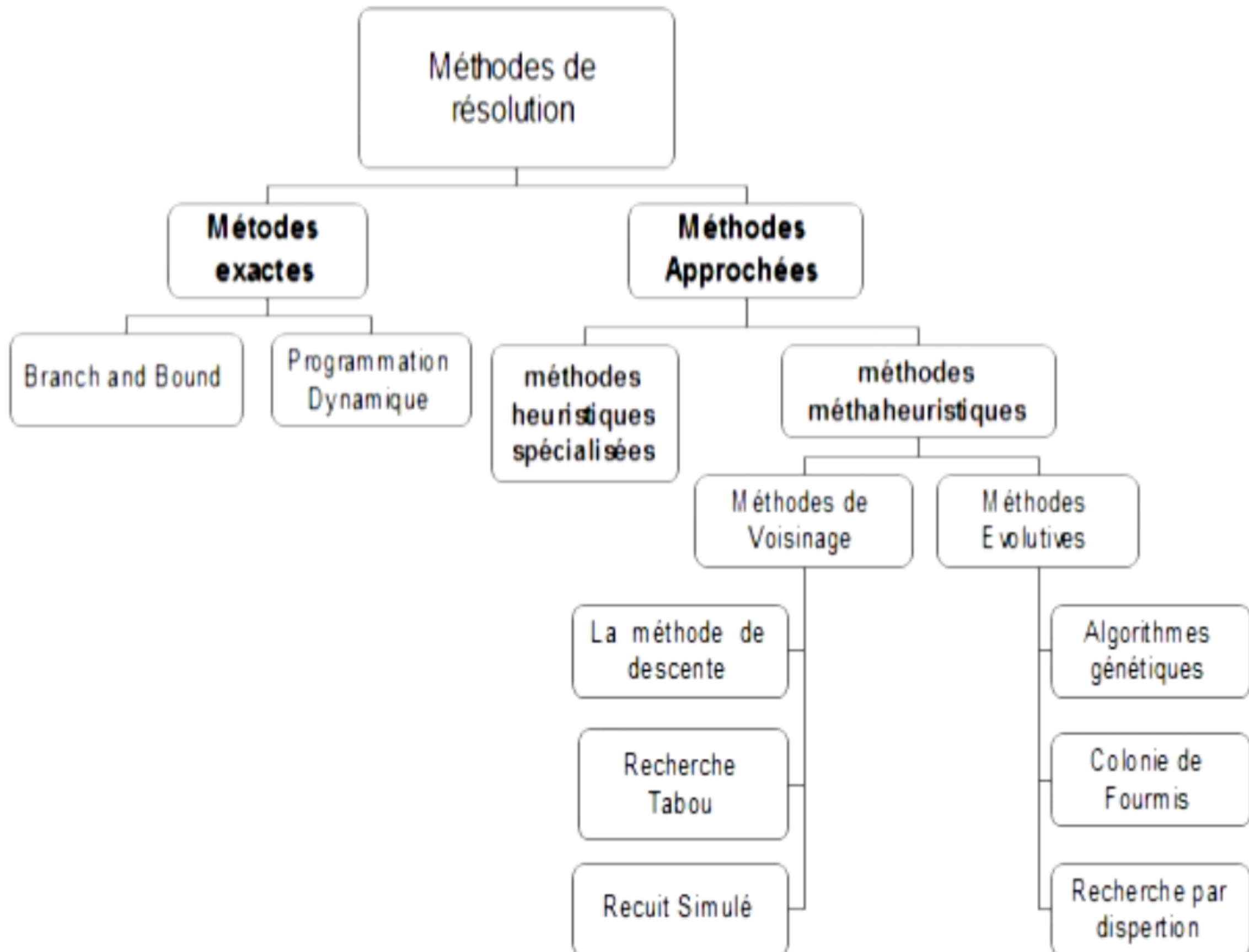
Méthodes de résolutions en OC

Deux classes de méthodes pour résoudre les problèmes d'optimisation combinatoire :

Méthodes exactes qui garantissent l'optimalité de la résolution mais qui perdent en efficacité pour les applications de grande taille. elles produisent une solution optimale pour une instance de problème d'optimisation donné. Elles se reposent généralement sur la recherche arborescente et sur l'énumération partielle de l'espace de solutions. Elles sont utilisées pour trouver au moins une solution optimale d'un problème. Les algorithmes exacts les plus connus sont la méthode de séparation et évaluation, la programmation dynamique, et la programmation linéaire

Méthodes approchées qui gagnent en efficacité et permettent de trouver des solutions de bonnes qualité en un temps de calcul raisonnable sans garantir l'optimalité.

Classification des méthodes de résolution



Programmation Mathématique

Programmation mathématique

Un Programme Mathématique (mathematical program), noté PM, est un problème d'optimisation sous contrainte (P) qui peut s'écrire de la façon suivante :

$$\begin{aligned} &\text{Maximiser } f(x) \\ &\text{sous les contraintes} \\ &g_i(x) \leq 0 \qquad i = 1, \dots, m \\ &x \in S. \end{aligned}$$

- S est une partie de $\mathbb{R}^n$ et x est un vecteur appelé variable, ces n composantes sont dites les inconnues du problème.
- la fonction $f : S \rightarrow \mathbb{R}$ est appelée fonction objective ou objectif
- les fonctions $g_i : S \rightarrow \mathbb{R}$, $i = 1, \dots, m$, forment des inégalités qui sont appelées les contraintes du problème.

Programmation mathématique

$$\begin{aligned} &\text{Maximiser } f(x) \\ &\text{sous les contraintes} \\ &g_i(x) \leq 0 \qquad i = 1, \dots, m \\ &x \in S. \end{aligned}$$

- si la fonction objective f et les fonctions contraintes g sont toutes linéaires : on parle de programme linéaire.
- si l'ensemble S est continu, on parle de programme mathématique continu.
- si l'ensemble S est discret (c'est-à-dire isomorphe à $\mathbb{N}^n$), on parle de programme mathématique discret ; on le dit également entier si $S \subset \mathbb{N}^n$ ou même binaire si $S \subset \{0,1\}^n$
- si certaines composantes du vecteur x solution prennent leurs valeurs dans un ensemble discret et les autres dans un ensemble continu, on le dit programme mathématique mixte.

Programmation mathématique

- Exemple d'un programme mathématique linéaire en variable 0 - 1

$$\begin{array}{ll}\max & \sum_{i=1}^n c_i x_i \\s.c. & \sum_{i=1}^n a_{ki} x_i \leq b_k, \quad k = 1, \dots, m \\ & x_i \in \{0, 1\} \quad i = 1, \dots, n\end{array}$$

Problèmes classiques en optimisation combinatoire

Problème du sac à dos

On dispose d'un sac-à-dos dont le contenu ne peut pas excéder une capacité c . Considérons un ensemble de n objets, numérotés par l'indice i de 1 à n , possédant chacun un poids w_i (weight en anglais) et une valeur p_i (profit en anglais).

Le problème du sac-à-dos consiste à trouver le sous-ensemble d'objets à charger dans le sac de capacité c afin de **maximiser** la somme des profits.

Ce problème se rencontre bien entendu dès que l'on part en randonnée en voulant emmener le plus possible d'objets utiles (nourriture, boissons,...). Mais ce problème est plus fréquemment utilisé pour remplir les camions de transport, les avions ou bateaux de fret et même pour gérer la mémoire d'un microprocesseur

Problème du sac à dos à variable binaire

Variables de décision : Les variables sont définies sur l'ensemble 0, 1 : La variable x_i du i -ème élément, $x_i = 1$ si l'élément i est mis dans le sac, $x_i = 0$ s'il est laissé de côté.

Contraintes : Le problème sac-à-dos est limité par deux contraintes. En plus de la contrainte d'intégrité $x_i \in 0, 1$, le sac-à-dos ne doit pas dépasser sa capacité c (volume ou poids).

Critère : La fonction objectif du problème sac-à-dos se définit comme la maximisation des profits des objets mis dans le sac.

$$\begin{array}{ll} \text{Maximiser} & f(x) = \sum_{i=1}^n p_i \cdot x_i \\ \text{s.c.} & \end{array}$$

$$x_i \in \{0, 1\}, \quad i \in \{1, \dots, n\}$$

$$\sum_{i=1}^n w_i \cdot x_i \leq c$$

Problème de voyageur de commerce (TSP)

Un agent commercial doit visiter une liste de villes et retourner à sa ville de départ. Cette tournée entre toutes les villes doit prendre le chemin le plus court. Le plus court chemin peut se traduire par la distance, le temps ou même le carburant.

Chaque ville doit être visitée une et une seule fois.

Pour chaque paire de ville, on connaît la distance entre les deux villes.

Formalisme du (TSP)

Les n villes sont représentées par un graphe complet de n sommets, valué par les distances entre ces villes. On note ce graphe $G = (X, U, c)$ où $X = \{1, \dots, n\}$ est l'ensemble de sommets (de villes), U est l'ensemble d'arêtes et c est une fonction de coût qui représente la distance entre les 2 villes (c_{ij} est la distance entre les villes i et j)

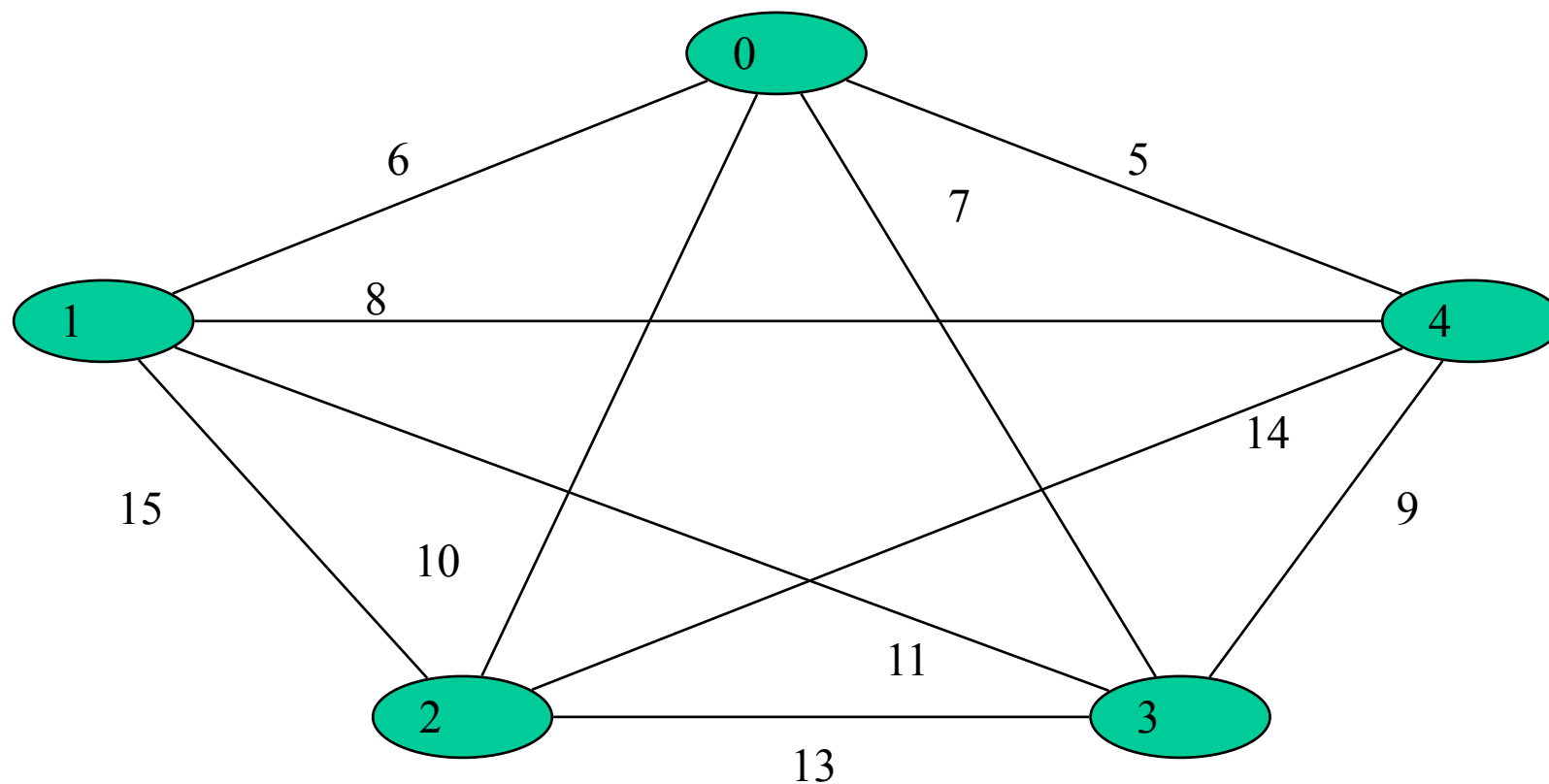
Le problème du voyageur de commerce (TSP, Travelling Salesman Problem) consiste à trouver le cycle de longueur minimale ² passant par chaque ville une et une seule fois : cycle hamiltonien (appelé aussi tour ou tournée)

Pour un nombre de n villes, il existe $n!$ chemin possible : **explosion exponentielle**. Pour $n = 20$ l'énumération des affectations (trajets) possibles, à vitesse d'un million par seconde, prendrait 77 000 ans !

TSP symétrique ou asymétrique

Le graphe représentant le problème de voyageur de commerce peut être orienté ou non-orienté. Il est orienté si la distance entre les deux villes n'est pas la même pour l'aller et le retour ($c_{ij} \neq c_{ji}$). Dans ce cas, on parle de problème de voyageur de commerce asymétrique.

Un problème de voyageur de commerce symétrique est défini par un graphe non-orienté (c'est-à-dire $c_{ij} = c_{ji}$)



Modélisation binaire du TSP symétrique

Variables de décision : Les variables sont définies sur l'ensemble $\{0, 1\}$: La variable x_{ij} correspond à la présence de l'arc ij dans le cycle hamiltonien du voyageur. On note $x_{ij} = 1$ si le voyageur passe par l'arc existant entre les villes i et j , sinon $x_{ij} = 0$

Contraintes : Le problème de voyageur de commerce est limité par plusieurs contraintes :

- (a) Contrainte d'intégrité : $x_{ij} \in \{0, 1\}$
- (b) Le voyageur doit entrer une seule fois dans une ville
- (c) Le voyageur doit sortir une seule fois d'une ville

Critère : La fonction objectif du problème de voyageur de commerce se définit comme la minimisation de la longueur du chemin

Modélisation binaire du TSP symétrique

Variables de décision : Les variables sont définies sur l'ensemble $\{0, 1\}$: La variable x_{ij} correspond à la présence de l'arc ij dans le cycle hamiltonien du voyageur. On note $x_{ij} = 1$ si le voyageur passe par l'arc existant entre les villes i et j , sinon $x_{ij} = 0$

Contraintes : Le problème de voyageur de commerce est limité par plusieurs contraintes :

- (a) Contrainte d'intégrité : $x_{ij} \in \{0, 1\}$
- (b) Le voyageur doit entrer une seule fois dans une ville
- (c) Le voyageur doit sortir une seule fois d'une ville

Critère : La fonction objectif du problème de voyageur de commerce se définit comme la minimisation de la longueur du chemin

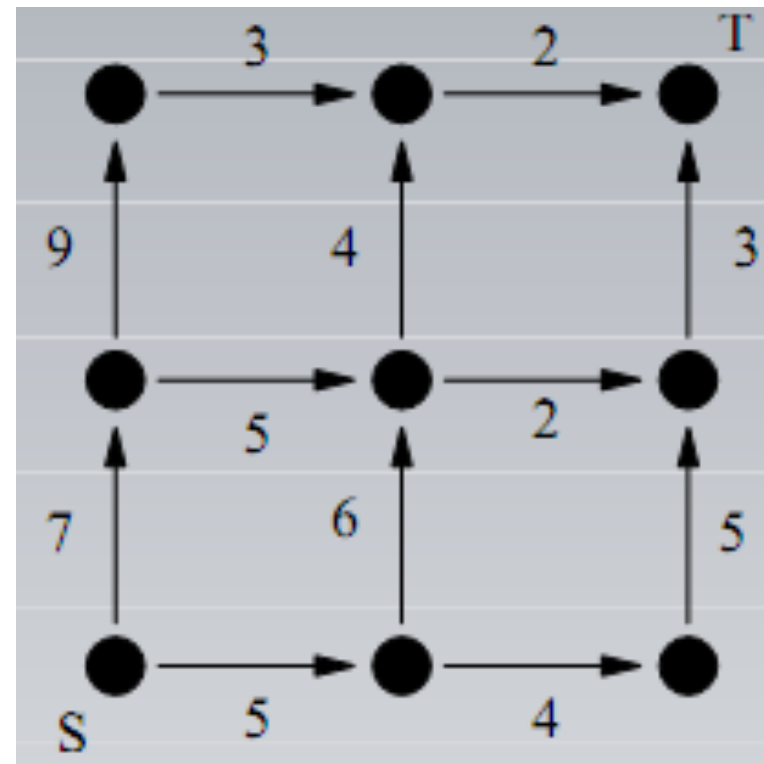
$$\min \sum_{(i,j) \in U} c_{ij} x_{ij}$$

$$\sum_{j:(j,i) \in U} x_{ji} = 1 \quad \forall i \in U$$

$$\sum_{j:(i,j) \in U} x_{ij} = 1 \quad \forall i \in U$$

Problème de plus court chemin

Trouver le plus court chemin entre deux sommets dans un graphe



Quelle est la taille de l'espace des solutions?

Faut-il parcourir toutes les solutions pour trouver la meilleure?

$$|X| = C_{n-1}^{2(n-1)} \text{ (pour } n = 50, |X| \sim 10^{30} \text{). Algorithme : } O(n^2)$$

Méthode du simplexe Primal et Dual

Rappel

Programme linéaire : problème d'optimisation de n variables de décision réelles où la fonction objectif est linéaire et où les contraintes sont également linéaires (équations ou inéquations).

Solution réalisable ou admissible : C'est un point $x=(x_1, \dots, x_n)$ qui vérifie les contraintes du problème.

Domaine réalisable : C'est l'ensemble des solutions réalisables.

Solution optimale : c'est une solution réalisable qui minimise (ou maximise) la fonction objectif sur le domaine réalisable.

Un PL Standard

Un programme linéaire est dit **standard** s'il est de la forme

$$\begin{array}{ll}\text{Max} & z = \sum_{j=1}^n c_j x_j \\ \text{s.c.} & \sum_{j=1}^n a_{ij} x_j = b_i \quad i = 1, \dots, m \\ & x_j \geq 0 \quad j = 1, \dots, n\end{array}$$

- Problème de maximisation
- Toutes les variables sont **non négatives**
- Toutes les autres contraintes sont des équations

$$\text{Formulation matricielle : } \begin{cases} \text{Max} & z = \mathbf{c}\mathbf{x} \\ \text{s.c.} & \mathbf{A}\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{cases}$$

Un PL Standard

Soit $\{Ax = b, x \geq 0\}$ le système de contraintes d'un P.L. standard avec $A : m \times n$ et $\text{rang}(A) = m$.

- Une base B du P.L. est une base du système $Ax = b$.
- La solution particulière

$$x_B = B^{-1}b \quad \text{et} \quad x_N = 0$$

est appelée la solution de base (ou solution basique) associée à la base B .

- Dans la base B , les variables x_B sont dites basiques alors que les variables x_N sont dit hors base. De même, les colonnes de B sont dites basiques et celles de N hors base.
- La solution de base associée à B s'obtient en fixant à zéro les **variables hors base**

Tableau associé à la base B

$B^{-1}A$	$B^{-1}b$
$c^T - c_B^T B^{-1}A$	$-c_B^T B^{-1}b$

Le pivotage

Dans un système $Ax=b$, un pivotage est ensemble d'opérations de ligne élémentaires permettant de rendre une ligne basique (passer du tableau d'une base B au tableau d'associé à une base voisine de B).

Plus précisément, pivoter autour du pivot a_{ir} (non nul), consiste à :

- La ligne i du pivot est divisée par le pivot a_{ir} (pour isoler la variable x_r dans la $i^{\text{ème}}$ ligne)
- Eliminer x_r des autres équations par substitution : Pour toute ligne k autre que la ligne du pivot, on soustrait de ligne k la nouvelle ligne du pivot (après division par le pivot) multipliée par le coefficient de cette ligne sur la ligne r du pivot. ($i' = i - k' \cdot \text{coef}$)
- La ligne ayant un 0 sur la colonne du pivot, ne change pas.

Algorithme du simplexe primal : principe

L'algorithme du simplexe (primat) (G. B. Dantzig, 1947) est un algorithme itératif permettant de résoudre un problème de programmation linéaire.

Dans sa version la plus classique, la méthode compte deux phases :

- ◆ **la phase II** : A partir d'une solution de base admissible du P.L., l'algorithme cherche une base admissible voisine dont la solutionnée base est meilleure. la méthode s'arrête :
 - ◆ soit lorsqu'il n'est plus possible d'améliorer la valeur de la solution de base courante qui est alors optimale,
 - ◆ soit lorsqu'un rayon extrême du domaine admissible est découvert, le long duquel la valeur de la fonction objectif augmente à l'infini, le problème est alors non borné.
- ◆ **la phase I** : Cette phase permet de trouver une solution de départ admissible ou de prouver qu'il n'en existe pas (le problème étant alors sans solutions admissibles - domaine réalisable vide)

Algorithme du simplexe primal

Étape 0 : On forme le tableau initial

B	x_1	x_2	$\dots$	x_n	x_{n+1}	x_{n+2}	$\dots$	x_{n+m}	
x_{n+1}	a_{11}	a_{12}	$\dots$	a_{1n}	1	0	$\dots$	0	b_1
x_{n+2}	a_{21}	a_{22}	$\dots$	a_{2n}	0	1	$\dots$	0	b_2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
x_{n+m}	a_{m1}	a_{m2}	$\dots$	a_{1n}	0	0	$\dots$	1	b_m
	c_1	c_2	$\dots$	c_n	0	0	$\dots$	0	0

La base initiale sera $(x_{n+1}, x_{n+2}, \dots, x_{n+m})$ ce qui correspond au point de départ $(0, 0, \dots, 0)$.

Étape 1 : On choisit la colonne de pivot (variable d'entrée)

Pour cela, on choisit l'indice j tel quel : $c_j = \max(c_i / c_i < 0)$

Si aucun choix n'est possible, on a atteint la solution optimale et l'algorithme se termine. Sinon, on passe à l'étape suivante.

Algorithme du simplexe primal

Étape 2 : On choisit la ligne de pivot (variable de sortie).

Pour cela, on choisit l'indice i en utilisant le critère des quotients caractéristiques :

$$\frac{b_i}{a_{ij}} = \min \left\{ \frac{b_k}{a_{kj}} \mid a_{kj} > 0 \quad k = 1, 2, \dots, m \right\}$$

où j est la colonne de pivot de l'étape 1

S'il n'existe pas de variable sortante, alors STOP, le problème est non borné

Étape 3 : mettre à jour la base et le tableau

pivoter autour de a_{ij} et revenir à l'étape 1

Algorithme du simplexe primal

Signature d'un tableau optimal :

		X_N				X_B				
B		x_1	x_2	$\dots$	x_n	x_{n+1}	x_{n+2}	$\dots$	x_{n+m}	
X_B	x_{n+1}					1	0	$\dots$	0	+
	x_{n+2}					0	1	$\dots$	0	+
	$\vdots$					$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
	x_{n+m}					0	0	$\dots$	1	+
	z	+	+	$\dots$	+	0	0	$\dots$	0	0

Signature d'un tableau non borné :

		X_N				X_B				
B		x_1	x_2	$\dots$	x_n	x_{n+1}	x_{n+2}	$\dots$	x_{n+m}	
X_B	x_{n+1}		-			1	0	$\dots$	0	+
	x_{n+2}		-			0	1	$\dots$	0	+
	$\vdots$		$\vdots$			$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
	x_{n+m}		-			0	0	$\dots$	1	+
	z		-			0	0	$\dots$	0	0

Programmation mathématique et dualité

Problème primal et problème dual

Problème de programmation linéaire avec inégalité

Problème primal

$$\begin{array}{ll}\min & c^T x \\ \text{Sujet à} & Ax \geq b \\ & x \geq 0\end{array}$$

Problème dual

$$\begin{array}{ll}\max & b^T y \\ \text{Sujet à} & A^T y \leq c \\ & y \geq 0\end{array}$$

y

x

Problème de programmation linéaire sous forme standard

Problème primal

$$\begin{array}{ll}\min & c^T x \\ \text{Sujet à} & Ax = b \\ & x \geq 0\end{array}$$

Problème dual

$$\begin{array}{ll}\max & b^T y \\ \text{Sujet à} & A^T y \leq c\end{array}$$

y

x

Règles de duplication

Problème de maximisation



Problème de minimisation

Variable $x_j \geq 0$



j^{e} contrainte de type $\geq$

Variable x_j libre



j^{e} contrainte de type $=$

Variable $x_j \leq 0$



j^{e} contrainte de type $\leq$

j^{e} contrainte de type $\leq$



Variable $y_i \geq 0$

j^{e} contrainte de type $=$



Variable y_i libre

j^{e} contrainte de type $\geq$



Variable $y_i \leq 0$

Algorithme dual du simplexe

- L'algorithme dual du simplexe est une méthode itérative pour résoudre un problème de programmation linéaire sous sa forme standard

$$\min \quad z = c_1x_1 + c_2x_2 + \dots + c_nx_n$$

Sujet à

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2$$

$$\cdot \quad \cdot \quad \cdot \quad \cdot$$

$$\cdot \quad \cdot \quad \cdot \quad \cdot$$

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m$$

$$x_j \geq 0 \quad j = 1, 2, \dots, n$$

- À chaque itération nous avons une **solution de base du problème qui n'est pas réalisable**, sauf à la dernière itération de l'algorithme, et pour laquelle **les coûts relatifs de toutes les variables sont non négatifs** (signature de solution optimale)

Algorithme dual du simplexe

- Etape 0 : une itération typique de l'algorithme s'effectue sur le tableau du simplexe associé à la solution de base courante :

Var. base	x_1	x_2	...	x_{j_r}	...	x_{j_1}	...	x_s	...	x_{j_m}	...	x_n	$-Z$	Termes droite
x_{j_1}	$\bar{a}_{11}$	$\bar{a}_{12}$	...	0	...	1	...	$\bar{a}_{1s}$	...	0	...	$\bar{a}_{1n}$	0	$\bar{b}_1$
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
x_{j_r}	$\bar{a}_{r1}$	$\bar{a}_{r2}$	...	1	...	0	...	$\bar{a}_{rs}$	...	0	...	$\bar{a}_{rn}$	0	$\bar{b}_r$
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
x_{j_m}	$\bar{a}_{m1}$	$\bar{a}_{m2}$	...	0	...	0	...	$\bar{a}_{ms}$	...	1	...	$\bar{a}_{mn}$	0	$\bar{b}_m$
$-Z$	$\bar{c}_1$	$\bar{c}_2$	...	0	...	0	...	$\bar{c}_s$	...	0	...	$\bar{c}_n$	1	$-\bar{z}_0$

$$\begin{aligned} \bar{c}_j &\geq 0 \quad \forall j = 1, 2, \dots, n \\ \bar{c}_{j_i} &= 0 \quad \forall i = 1, 2, \dots, m \end{aligned}$$

- Etape 1 : Si $\bar{b}_i \geq 0 \quad \forall i = 1, 2, \dots, m$, alors la solution est réalisable et optimale. L'algorithme se termine.

- Etape 2 : Sinon soit $\bar{b}_r = \min_{1 \leq i \leq m} \{\bar{b}_i\} < 0$. Si $\bar{a}_{rj} \geq 0 \quad \forall j = 1, 2, \dots, n$, alors le problème n'est pas réalisable. En effet puisque

$$\sum_{j=1}^n \bar{a}_{rj} x_j \geq 0 \quad \text{et} \quad \bar{b}_r < 0$$

il est impossible que $\sum_{j=1}^n \bar{a}_{rj} x_j = \bar{b}_r$.

Algorithme dual du simplexe

- Etape 3 : choix de la variable de sortie

Sinon soit $\bar{b}_r = \min_{1 \leq i \leq m} \{\bar{b}_i\} < 0$. x_{j_r} est la variable de sortie.

Le pivot se fera dans la ligne r du tableau.

- Etape 4 : choix de la variable d'entrée

L'indice s de la variable d'entrée est tel que :

$$\frac{\bar{c}_s}{\bar{a}_{rs}} = \max_{1 \leq j \leq n} \left\{ \frac{\bar{c}_j}{\bar{a}_{rj}} : \bar{a}_{rj} < 0 \right\} \quad \text{ou} \quad \frac{\bar{c}_s}{-\bar{a}_{rs}} = \min_{1 \leq j \leq n} \left\{ \frac{\bar{c}_j}{-\bar{a}_{rj}} : \bar{a}_{rj} < 0 \right\}$$

Pour retrouver le tableau du simplexe associé à la nouvelle base où la variable d'entrée x_s remplace la variable de sortie x_r , il suffit de faire un pivot sur l'élément $\bar{a}_{rs} < 0$ et revenir à l'étape 1

Algo. primal du simplexe VS Algo. dual du simplexe

Algo. du simplexe

Recherche dans le domaine réalisable

Choisit la variable d'entrée pour réduire la valeur de la fonction économique

Choisit la variable de sortie pour préserver la réalisabilité

Stop quand une solution optimale est trouvée ou que le problème n'est pas borné inférieurement

Algo. dual du simplexe

Recherche à l'extérieur du domaine réalisable

Choisit le variable de sortie pour éliminer une variable de base négative

Choisit la variable d'entrée pour préserver la condition d'optimalité

Stop quand la solution est réalisable ou quand le problème n'est pas réalisable

Vecteur multiplicateur du simplexe

$$\bar{c}^T = c^T - c_B^T B^{-1} A$$

Dénotons le vecteur $\pi \in R^m$ défini par $\pi^T = c_B^T B^{-1}$

π est le vecteur des multiplicateurs *du simplexe associé à la base B*.

Alors $\bar{c}^T = c^T - \pi^T A$

ou $\bar{c}_j = c_j - \pi^T a_{\bullet j}$

où $a_{\bullet j}$ dénote la $j^{\text{ième}}$ colonne de la matrice de contrainte A